

Track 1 Practice Set

Python → NumPy → Pandas, graded easy to hard — pharma-framed, for the RWE/Clinical Data Analyst path

The one rule that makes this count

After each exercise, commit it: `git add . && git commit -m "pandas: groupby on diabetes outcome"` — one commit per exercise. A notebook you ran once and never committed is invisible. Commits are your proof of progress, not the exercise itself.

Repo suggestion

One repo `rwe-practice`, three folders: `01_python/`, `02_numpy/`, `03_pandas/`. Each exercise = one `.ipynb` or `.py` file.

Datasets to download first

#	Dataset	Kaggle slug / search	Used in
A	Pima Indians Diabetes	<code>uciml/pima-indians-diabetes-database</code>	Pandas P1–P6
B	UCI ML Drug Review	<code>jessicali9530/kuc-hackathon-winter-2018</code>	Pandas P7–P11
C	Medical Cost Personal (optional)	search “Medical Cost Personal Datasets”	stretch

Dataset A is clean and numeric — start there. Dataset B is messier (text, dates, categories) — that mess is exactly what RWE work looks like, so it's the more valuable one once you're comfortable.

Tier 1 — Python Fundamentals

No dataset, pure logic. You've done the dose calculator, so these push just past it. Move fast — this is mostly revision.

PY1 — Prescription label formatter

Given `drug = "Amoxicillin"`, `dose = 500`, `unit = "mg"`, `freq = 3`, print a clean label: Amoxicillin 500 mg – take 3 times daily. Use an f-string.

■ **Done when:** *output is exactly formatted, no trailing spaces.*

PY2 — Weight list, manual stats

`weights = [58, 72, 65, 90, 47, 83]` (kg). Without NumPy, compute and print mean, max, min, and the range (max–min). Use `sum()`, `len()`, `max()`, `min()`.

■ **Done when:** *mean prints as a float, range is correct.*

PY3 — Drug → standard dose dictionary

Build `doses = {"amoxicillin": 500, "paracetamol": 650, "ibuprofen": 400}`. Ask the user for a drug name, `.lower()` it, and print its dose. Handle "not found" with `.get()`.

■ **Done when:** *unknown drug prints a clean "not in list" message instead of crashing.*

PY4 — Flag over-limit doses (loop + conditional)

`daily_doses = [400, 900, 650, 1200, 300]` mg, `max_safe = 800`. Loop through, print each dose and whether it's SAFE or OVER LIMIT. Count how many are over.

■ **Done when:** *it prints the count of over-limit doses at the end.*

PY5 — BMI function with validation

Write `def bmi(weight_kg, height_m):` that returns BMI rounded to 1 decimal, and handles the case where height is 0 (avoid divide-by-zero). Test with 3 patients.

■ **Done when:** *calling with height=0 doesn't crash the program.*

PY6 — Cohort filter (functions + list)

Given a list of patient dicts `[{"id":1, "age":45, "bmi":31}, ...]` (make 6), write a function that returns only patients with `bmi >= 30`. This is a cohort selection — the exact logic you'll do in RWE, just without Pandas yet.

■ **Done when:** *returns a filtered list, original list unchanged.*

Tier 2 — NumPy (arrays + vectorized thinking)

The shift here: stop looping, start operating on whole arrays at once. That's the mental model Pandas is built on.

NP1 — Concentration array stats

`conc = np.array([1.2, 3.5, 2.8, 4.1, 0.9, 3.3])` (mg/L). Print mean, std, median, and the index of the max value (`np.argmax`).

■ **Done when:** *all four print correctly.*

NP2 — Unit conversion, vectorized

Same array in mg/L. Convert the whole array to mcg/L ($\times 1000$) in one line, no loop. Then round to 0 decimals with `np.round`.

■ **Done when:** *it's one vectorized operation, zero for-loops.*

NP3 — Boolean masking (threshold cohort)

From `conc`, use a boolean mask to pull only values above 3.0. Then count how many with `.sum()` on the mask.

■ **Done when:** *you get the sub-array AND the count, both from the mask.*

NP4 — 2D concentration–time matrix

Make a 4×3 array: 4 patients (rows) × 3 timepoints (columns). Compute the mean per patient (`axis=1`) and mean per timepoint (`axis=0`). Say out loud which axis is which — this trips everyone up.

■ **Done when:** *you can explain why axis=1 gives per-patient means.*

NP5 — np.where categorization

Given an age array, use `np.where` to label each as "adult" (≥ 18) or "minor". Then a nested `np.where` for three bins (child/adult/senior).

■ **Done when:** *the 3-bin version works.*

NP6 — Bridge to Pandas

Load ONE numeric column from the diabetes CSV using `np.genfromtxt(..., delimiter=',', skip_header=1, usecols=...)`. Compute its mean. (This is deliberately clunky — it shows you why Pandas exists.)

■ **Done when:** *you've felt the pain of doing this without Pandas.*

Tier 3 — Pandas (real datasets — the RWE core)

This is where the real value is. Spend your morning blocks here.

Dataset A: Pima Diabetes (P1–P6)

P1 — Load & inspect

`df = pd.read_csv("diabetes.csv")`. Run `.head()`, `.tail()`, `.shape`, `.info()`, `.describe()`. Write one sentence per method on what it told you.

■ **Done when:** *you can state how many rows/columns and which columns are numeric.*

P2 — Select & filter

Select just Glucose and Age. Then filter to patients with `Glucose > 140`. How many are there?

■ **Done when:** *you report the filtered row count.*

P3 (starred) — The "zero is really missing" problem

In this dataset, Glucose, BloodPressure, BMI, Insulin, SkinThickness have literal 0s that are impossible (a living person's BMI isn't 0). Count zeros per column, then replace them with NaN (`df[col].replace(0, np.nan)`), then count missing with `.isna().sum()`.

■ **Done when:** *you've quantified missingness — this is a core RWE data-quality task.*

P4 — groupby

`df.groupby("Outcome").mean()`. Compare mean Glucose, BMI, Age between diabetic (1) and non-diabetic (0). Write the takeaway in one line.

■ **Done when:** *you can state which features differ most between groups.*

P5 — Derived column (feature engineering)

Create `BMI_category` using `pd.cut` (bins: <18.5 underweight, 18.5–25 normal, 25–30 overweight, >30 obese). Then `value_counts()` on it.

■ **Done when:** *the category distribution prints.*

P6 — Correlation

`df.corr()["Outcome"].sort_values()`. Which features correlate most with diabetes outcome?

■ **Done when:** *you name the top 2.*

Dataset B: Drug Review (P7–P11) — messier, more realistic

P7 — Load & assess

It's tab-separated: `pd.read_csv("drugsComTrain_raw.tsv", sep="\t")`. Check `.shape`, `.info()`, `.isna().sum()`. This has ~160k rows — notice how `.head()` matters more when you can't eyeball everything.

■ **Done when:** *you report row count and which columns have missing data.*

P8 — value_counts (top drugs / conditions)

Top 10 most-reviewed drugName. Top 10 condition. Notice any junk values in condition (there are some malformed ones — flag them).

■ **Done when:** *you've spotted at least one dirty condition value.*

P9 (starred) — groupby + filter (the real analyst move)

Mean rating per drugName, BUT only for drugs with at least 50 reviews (otherwise averages are noise). Chain: `groupby` → `agg` (count & mean) → `filter` `count>=50` → `sort`. This "filter to adequately-sampled groups" pattern is RWE bread and butter.

■ **Done when:** *you have a clean table of well-reviewed drugs ranked by rating.*

P10 — Dates & trend

Parse the date column (`pd.to_datetime`), extract year (`.dt.year`), and count reviews per year. Is the data skewed to certain years?

■ **Done when:** *you have a reviews-per-year table.*

P11 (starred) — Aggregate & export (portfolio-worthy)

Build a summary table: for the top 20 conditions, show number of drugs reviewed, total reviews, and mean rating. Export to `condition_summary.csv`. This one goes in your GitHub README as a mini-analysis.

■ **Done when:** *the CSV exists and you've written 2–3 sentences interpreting it.*

Capstone (do this once P1–P11 are committed)

One notebook, `diabetes_eda.ipynb`, that: loads the diabetes data → cleans the zero-missingness → shows group differences by outcome → engineers BMI category → states 3 findings in plain English. This becomes your first real portfolio piece and the thing you point a recruiter to. When you hit Matplotlib next, add plots to **this same notebook** — don't start a new one.

Sequencing (honest version)

- Tier 1 is mostly revision — move fast, don't linger. If PY1–PY4 feel trivial, skip to PY5–PY6.
- Tiers 2–3 are the actual value. Spend your morning blocks here.
- P3, P9, P11 (starred) are the ones that look like real RWE work. If you only do three Pandas exercises this week, do those.
- Matplotlib slots onto these same two datasets once you finish P1–P6 (diabetes) — no new downloads needed. Do 2–3 charts on diabetes, then continue to P7–P11, then SQL.
- Commit as you go. "Almost done" is not a git status.